Reinforcement Learning for Dynamic Data Fragmentation and Allocation

Tinotenda Kaseke
Department of Computer Science
University of Texas Permian Basin
Odessa, TX, USA
tinokaseke@gmail.com

***Abstract*—Distributed databases fix their fragmentation schema when the database is designed and place the resulting shards with a function of the key, which never observes where requests come from. When the geographic centre of a workload moves, as it does daily in a multi-region service, the placement drifts out of step with the traffic and transaction latency degrades. We model continuous re-fragmentation and re-allocation as a Markov decision process over a simulated six-region cluster, with a reward in which migration cost is a fraction of the dataset rather than a byte count so that a policy trained in simulation can be scored on real hardware without the reward changing meaning. On a six-node CockroachDB cluster with wide-area latency injected between nodes, gathering leaseholders in the region issuing the transactions rather than spreading them changes p95 latency by 1372.5 ms, which establishes the premise outside simulation. Sweeping workload volatility over a factor of eight shows that a reactive heuristic sits 0.44 ms above the achievable bound under slow drift and 57.20 ms above it under fast drift, 41% of its own latency, whilst the data it moves rises from 405 GB to 1115 GB per episode; scored under the full objective at the highest setting it becomes indistinguishable from static hashing. A Deep Q-Network trained at that setting beats both baselines on held-out seeds, on two independent learner seeds, reaching a latency 11 ms worse than the heuristic whilst moving one twentieth of the data. The margin is narrow, the policy had not converged, and between 80% and 90% of the actions it selects are inapplicable, so the result is best read as evidence that the room the sweep identifies is reachable rather than as a solved problem.**

***Index Terms*—distributed databases, data fragmentation, data allocation, reinforcement learning, deep Q-networks, geo-distributed transactions**

I. Introduction

A distributed database splits its data into pieces and spreads those pieces across machines in different places. The pieces are usually called shards, or fragments, and the two decisions that matter are where the boundaries between them fall and which machine holds each one. Both decisions are made once, when the database is designed. The boundaries are fixed by a schema, and the placement is decided by a function of the key, such as a hash, which maps a record to a machine without ever considering where the requests for that record come from.

That arrangement works whilst the traffic stays where the designer expected it. In a service with users on several continents, served by a database spread across regions in the manner of Spanner [1] or CockroachDB [2], it does not. The geographic centre of the workload moves across the day as one region wakes and another sleeps, and it moves abruptly when something local draws attention to one region. A placement chosen for the morning is wrong by the evening. Every request that has to cross an ocean to reach its data pays the round-trip time of that ocean, and a transaction touching several shards held in several regions pays it repeatedly, because the protocol that keeps such a transaction atomic requires more than one round of agreement between the machines involved. The result is that latency degrades steadily as the mismatch between placement and traffic grows.

The obvious response is to move the data when the traffic moves. A simple rule does this: each decision window, find the shard whose placement costs the most and move it towards the region reading it. I implement that rule as a baseline and call it *follow traffic*. The question this paper asks is whether anything is gained by replacing it with a policy learned from experience, and the answer turns out to depend entirely on how fast the workload moves. When drift is slow, the simple rule is within half a millisecond of the best placement that moving shards can achieve, and there is nothing worth learning. When drift is fast, the same rule falls 57 ms short of that bound, which is 41% of its own latency, because it reacts one window behind a target that has already moved again. The room for a learned policy is therefore largest exactly where the problem is hardest, and that is a measured observation rather than an assumption.

This paper reports the following. I build a simulator of a six-region cluster with a workload whose geographic centre drifts on a daily cycle and spikes locally, and I define the placement problem on it as a Markov decision process with a flat action space of 261 declarative actions. I establish four reference points against which any learned policy must be judged: static hash partitioning, which is what production systems do today; a random control; the follow-traffic heuristic; and a per-window bound on what lease movement alone can achieve. I measure how the gap between the heuristic and that bound widens with workload volatility. I verify on a real six-node CockroachDB cluster, with wide-area latency injected between the nodes, that placement changes transaction latency by far more than the measurement noise, which is the precondition for any of this being learnable outside simulation. Finally I train a Deep Q-Network on the simulator and report where it stands: at volatility 4 it beats both the heuristic and static hashing on the objective, on two independent learner seeds,

by moving a twentieth of the data the heuristic moves; and it remains a long way from the bound, with most of the actions it selects inapplicable in the state it selects them in. Fig. 1 shows how the simulator, the agent and the cluster fit together, and which three things have to agree between them for a policy trained in one to mean anything in the other.

II. Related Work

Three bodies of work meet in this problem: the classical treatment of fragmentation and allocation as an optimization to be solved, recent systems that adapt placement to a moving workload, and the use of learning to replace hand-written database administration.

A. Fragmentation and allocation as an optimization problem

The question of how to divide a database and where to put the pieces is as old as distributed databases themselves, and the classical treatment is to state it as an optimization problem and solve it once. A cost model scores a candidate arrangement against an expected workload, and a solver searches for the arrangement that minimizes it. The result is then installed and the database runs with it.

The literature divides the problem in two. Fragmentation asks how to cut the data; allocation asks where to put the pieces. Ceri et al. [3] treat horizontal fragmentation, dividing a relation by rows, and Navathe et al. [4] treat vertical fragmentation, dividing it by columns so that a fragment matches the attributes a transaction actually uses. Both optimize against an expected access pattern supplied in advance. On the allocation side, Apers [5] formalizes the placement of fragments across network sites as a cost minimization and analyzes its complexity, Saccà and Wiederhold [6] give a greedy construction for a cluster of processors, and March and Rho [7] extend the objective to place operations as well as data, under capacity constraints. The standard synthesis of this material is the textbook treatment of Özsu and Valduriez [8], and Nashat and Amer [9] give a taxonomy of the techniques that followed. The formulation remains current: Amer et al. [10] solve fragmentation and allocation together as one heuristic optimization, and Niu et al. [11] state the allocation problem as a balanced cut.

A second tradition avoids the optimization entirely. Consistent hashing [12] assigns a key to a node through a function of the key alone, so that adding or removing a node moves only a small share of the data and no central directory is needed. CRUSH [13] extends that to replicated data under user-defined failure domains, and a recent survey compares the family [14]. This is what the static baseline in this paper is, and it is what most production systems use. It is also the root of the problem studied here: a function of the key cannot observe where a request came from.

The weakness is in the word once. A solved-once answer is correct for the workload it was solved against and degrades as the real workload departs from it. Where the departure is slow this is tolerable, and where the workload has a geographic centre that moves on a daily cycle it is not. That observation motivates the systems in the next group, which re-solve the problem as the workload changes.

B. Adapting placement to a moving workload

A line of work makes the partitioning decision from the observed workload rather than from a design-time estimate. Schism [15] builds a graph in which tuples accessed by the same transaction are joined by an edge, then partitions that graph to minimize distributed transactions whilst keeping partitions balanced. Pavlo et al. [16] search the design space with an analytical cost model that accounts for skew as well as coordination cost. E-Store [17] addresses the case where a small number of tuples carry most of the traffic, placing hot ranges individually whilst distributing cold data in large chunks. Clay [18] removes the restriction to hierarchical schemas, migrating blocks built around hot tuples together with the tuples co-accessed with them. Kamal et al. [19] make the repartitioning incremental, recutting only the subgraph that distributed transactions touch so that the scheme is refreshed as the workload drifts rather than recomputed. Promi [20] addresses the migration itself rather than the decision, moving data at a finer granularity so that rebalancing does not stop transactions.

A related line treats placement and replication as one problem rather than two stages. Atrey et al. [21] make that argument directly for geo-distributed clouds, solving placement and replication jointly by overlapping correlation clustering, and Adnan et al. [22] optimize placement against network cost in the same setting. Both reach the joint framing this work uses, and both reach it with a solver rather than a learned policy.

These systems share the premise of this work, that placement should follow the workload, and they share a limitation. Each re-solves an optimization problem when a trigger fires, using a cost model written by hand. None improves from the outcomes of its own past decisions, and none of their cost models carries a term for geographic distance, because they were designed for clusters in one data centre where the distance between any two nodes is much the same.

Lion [23] is the closest system to this work in intent. It observes that a transaction is slow mainly because it spans machines, and it therefore arranges replicas so that some single node holds a copy of every partition a given transaction touches, allowing that transaction to commit locally rather than through the multi-round agreement protocol that a distributed commit requires. A graph-based analyzer groups partitions that are accessed together, a cost model assigns those groups to nodes subject to a load-balance constraint, and an LSTM predictor forecasts the workload so that replicas are provisioned before the shift rather than after it. Lion reports up to 2.7 times higher throughput and 76.4% better scalability than the approaches it is compared against.

Lion establishes three things this work depends on. Adapting placement to a moving workload is worth a large amount of real throughput. Migration cost and load balance both belong in the objective rather than latency alone. Anticipating a shift

beats reacting to one, by roughly a factor of four in adaptation speed.

It also leaves three openings. The adaptation is a heuristic pipeline rather than a learned policy: the LSTM forecasts, but a hand-written cost model decides, so the system never improves from the outcomes of its own decisions. Geographic distance does not appear in its cost function, which is expressed in bytes moved, so two placements that differ by an ocean score identically. Fragment boundaries are fixed, so Lion moves replicas of a partition but never splits one.

GeoTP [24] supplies the geographic dimension that Lion does not have. It targets database middleware connecting heterogeneous data sources across regions, where the dominant costs are the network latency between the middleware and its data sources and the length of time a transaction holds locks whilst waiting on remote work. GeoTP reduces both: a decentralized prepare phase cuts the number of network round trips a distributed transaction needs, and a latency-aware scheduler delays lock acquisition so that the contention window is shorter. It reports up to 17.7 times improvement on YCSB and TPC-C.

GeoTP shows that latency awareness is worth building a scheduler around, and its cost accounting is the model for the one used here. It optimizes the execution of transactions against a given placement, however, and does not change where the data sits. The two papers are therefore complementary, and the gap between them is where this work sits: Lion adapts placement without seeing geography, GeoTP sees geography without adapting placement.

C. Learning to administer a database

A separate line of work replaces hand-written database administration with learned policies. In configuration tuning, CDBTune [25] treats the setting of database knobs as a reinforcement learning problem and learns from measured performance rather than from a regression over collected samples, and QTune [26] extends that to tune per query rather than per workload; the survey of Zhao et al. [27] covers the area. In query optimization, Neo [28] learns to produce execution plans and improves from the plans it has run. UDO [29] spans both, tuning index selection and system parameters together and distinguishing parameters that are expensive to change from those that are cheap, which is the same distinction this work draws between moving a lease and moving a replica.

The common argument across these systems is that the decision is made repeatedly against a changing workload, that its quality is measurable after the fact, and that a system able to learn from its own outcomes should eventually beat a fixed rule written by someone who could not anticipate every workload.

Placement has received less attention within this family than tuning or optimization. RLRP [30] applies a Deep Q-Network to replica placement and migration in object storage, addressing heterogeneous nodes and balancing load against movement. Sibyl [31] places data across hybrid storage devices with online reinforcement learning, and Harmonia [32] extends that to coordinate placement and migration as two agents. Zheng et al. [33] place replicas in an edge–cloud setting where geography does enter the objective, alongside reliability and load, and Liao et al. [34] learn placement against a latency objective inside a data centre. Closest of all in problem statement, Mahmoud et al. [35] formulate fragment allocation and replication as a Markov decision process and solve it with tabular Q-learning, driven by read, write and co-access matrices.

These differ from the present work in one or both of two ways. Most place data across *devices or tiers* that differ in speed, or across edge nodes that differ in capacity, rather than across regions that differ in distance from the traffic. And all of them hold the unit of placement fixed: a fragment, an object or a block is placed, never split. Mahmoud et al. [35] come closest on the first axis and still treat fragments as given, and a tabular Q-function does not extend to the state space a six-region cluster with twenty shards produces.

D. What is new here

This work learns fragmentation and geographic placement *jointly*, as one policy, for transactional workloads. The workload-driven partitioners re-solve a hand-written cost model and do not learn from their own outcomes. Lion learns a forecast but not a policy, and its cost model is denominated in bytes rather than in distance. GeoTP optimizes execution against a fixed placement. The learned administration systems address knobs, plans and indexes. The learned placement systems address storage tiers and edge nodes, not a transactional workload whose centre of gravity moves, and they hold the unit of placement fixed.

Stating the contribution precisely: the agent chooses, in one action space, where a shard's primary copy sits, where its replicas sit, and where the boundary between two shards falls, and it is rewarded on a cost that combines transaction latency, the volume of data moved, and load imbalance.

Two statements about that contribution, both deliberate. First, it is not claimed to be new. Searches of the indexes listed in `docs/literature-log.md` returned no paper that learns fragmentation and geographic placement jointly for transactional workloads; IEEE Xplore and the ACM Digital Library were reached only through aggregators, so the search does not reach as far as a novelty claim would require. Second, the agent described here is trained and measured, and on latency alone it does not outperform the hand-written heuristic it is meant to replace. What this paper establishes is the problem, the measurement apparatus, the baselines, and the conditions under which a learned policy has room to win.

III. System Model and Methodology

A. What is being decided

A cluster has R regions, each holding one database node. The keyspace is divided into C atomic cells, and a *shard* is a contiguous run of cells. Each shard has one *primary* copy, which serves writes, and some number of *replicas*, which hold the same data and can serve reads. The number of replicas per

shard, called the replication factor, is fixed at three throughout this work and is not something the agent controls.

Three decisions are therefore available at any moment. Which region holds a given shard's primary. Which regions hold its replicas. Where the boundary between two shards falls. The first two are allocation and the third is fragmentation, and this work treats them as one problem because they interact: splitting a shard is only worth doing if the two halves are then placed differently.

B. The cost of a placement

Latency is computed analytically from a fixed matrix of round-trip times between regions. A read costs the round trip to its nearest copy. A write costs the round trip to the primary plus the time for a majority of replicas to acknowledge it, since a write is not durable until enough copies agree. A transaction touching several shards costs its member operations plus, if those shards' primaries sit in more than one region, an additional charge for the two-phase commit protocol that keeps it atomic.

That last term is the mechanism this work is built on. It is what makes placing together two shards that are used together worth something, and therefore what makes the fragmentation decision interact with the allocation decision.

C. The workload

The workload generator produces a stream of requests whose geographic origin moves. Each of the C cells sits at an angular position on a circle, as do the regions, and traffic for a cell originates predominantly from whichever regions sit near its angle. The whole angular field rotates over a daily period, which is what makes a placement chosen at one time wrong at a later one. Cell popularity follows a Zipf distribution, so a minority of cells carry most of the traffic, and localized spikes arrive as a Poisson process, multiplying one region's traffic for a short burst. The read and write mix and the skew parameter follow the YCSB conventions [36], so the workload is recognizable rather than invented, with geographic origin added as the dimension YCSB does not model.

Adjacent cells are given correlated but not identical angular positions by a random walk. This is the detail that makes splitting worthwhile: a wide shard spans divergent angles and therefore cannot be placed well anywhere, whilst its two halves can each be placed well.

A single parameter, *volatility*, scales the rotation speed, the spike rate and the spike size together, so one number moves the environment from placid to violent. It is the main axis the evaluation sweeps.

D. The decision problem

The problem is stated as a Markov decision process. The agent acts once per decision window.

a) Observation: The state is encoded as a fixed-length vector describing, for every shard, its size and the regions holding its primary and replicas; for every region, its share of requests and its storage occupancy; the recent history of data movement; the matrix of inter-region round-trip times; and a small set of scalars summarizing current cost.

b) Actions: The action space is flat and declarative, with 261 actions at $R = 6$ regions and a cap of 20 shards. Four kinds:

- MOVE_LEASE(f, r) makes region r the primary for shard f. It moves no data and is only legal where r already holds a replica.
- MOVE_REPLICA_TO(f, r) places a replica of shard f in region r, evicting the existing replica furthest from r so that the replication factor stays constant. It costs one shard's worth of movement.
- SPLIT(f) divides shard f at its midpoint.
- NOOP does nothing.

Two properties of this encoding matter. First, every action names a desired end state rather than an operation, so every action is defined in every state: an action that cannot be applied resolves to a small penalty rather than an error. This matters because a Deep Q-Network cannot mask illegal actions, so an action space with illegal members would need machinery that this one avoids. Second, the encoding was chosen by measurement rather than by taste. The destination-only form of MOVE_REPLICA_TO gives 261 actions against 861 for the naive source-and-destination form, with a higher fraction of actions applicable in a typical state, and a breadth-first search over the reachability graph confirms that no placement becomes unreachable.

c) Reward: The reward is the negative weighted sum of three normalized terms: mean latency divided by a scale constant, data moved as a fraction of the total dataset, and load imbalance. Expressing movement as a fraction of the dataset rather than in bytes is deliberate and necessary: the simulator holds 60 GB and the real cluster used for validation holds about 1 GB, and a reward denominated in bytes would score the same physical move sixty times differently in the two environments, making a policy pre-trained in one meaningless in the other.

E. Baselines

Four reference points. Three are policies and one is a bound.

- **Static hash** never moves anything. The initial placement is what a consistent-hash partitioner [12] produces, and it is held for the whole episode. This is what a production database does today and it is the thing to beat.
- **Random** selects a legal action uniformly. It is a control: it proves that any gain comes from choosing *where* to move data rather than from moving data at all.
- **Follow traffic** finds, each window, the single shard whose primary placement costs the most relative to the best available region, and moves it towards that region. It is the obvious rule an operator would write by hand, and it is myopic by construction, weighing no migration cost and anticipating no drift.
- **Best per window (fixed shards)** is not a policy. It is the lowest latency reachable by relocating every primary for free and instantly, each window, at the shard boundaries

the episode started with. No real policy can reach it, because it pays no movement cost and sees the window it is optimizing. It is also not a global optimum, because it never splits a shard, so a policy that splits can legitimately sit below it.

F. The agent

The agent is a Deep Q-Network. A Q-network is a function that, given the current state, estimates the long-run value of each available action; the policy is then to take the action with the highest estimate. The network here has two hidden layers of 256 units and is trained with the standard Stable-Baselines3 implementation [37] against a Gymnasium environment [38], following Mnih et al. [39].

The plain form is used deliberately, without the extensions that Rainbow [40] combines: double Q-learning to correct overestimation [41], a dueling head [42], or prioritized replay [43]. The question here is whether the problem is learnable at all, and establishing that with the simplest agent makes any later improvement attributable to the extension rather than to the combination. The evaluation reports where that choice costs something.

Two choices in the training setup are worth stating because they are easy to get wrong. Training resets onto a rotating set of 64 workload seeds rather than one, because an agent trained on a single realization learns that realization rather than the structure behind it. Evaluation uses a block of seeds disjoint from the training seeds and acts greedily rather than exploring, so the reported number is generalization rather than recall.

G. Metrics

Every result in this paper reports some subset of five quantities.

- **Mean latency**, in milliseconds, averaged over the windows of an episode.
- **p95 latency**, the value below which 95% of observations fall. It is reported alongside the mean because an average can hide a bad tail, and users experience the tail.
- **Data moved**, in gigabytes per episode. This is the price paid for whatever latency was bought.
- **Cross-region transaction fraction**, the share of transactions whose shards span more than one region and which therefore pay the two-phase commit charge.
- **Load imbalance**, a measure of how unevenly requests are spread across regions, included because a policy can lower latency by piling all the work onto one region, which is not a real improvement.

Results are reported as the mean over eight seeds with a 95% percentile bootstrap [44] confidence interval computed over those seeds. The bootstrap is used rather than a t interval because eight is a small sample and nothing guarantees the per-seed means are normally distributed.

H. Experimental apparatus

Every number reported here comes from a single script that writes one self-contained directory per run, holding the resolved configuration, the seed list, the commit the code was at, the package versions, the machine, the raw per-window record, the per-seed summary, and the exact command that reproduces the run. Runs execute on GitHub Actions rather than on a personal machine, so each is unattended and each records the hardware it received. This is reported because it is part of the method: a number that cannot be traced to the configuration that produced it cannot be defended.

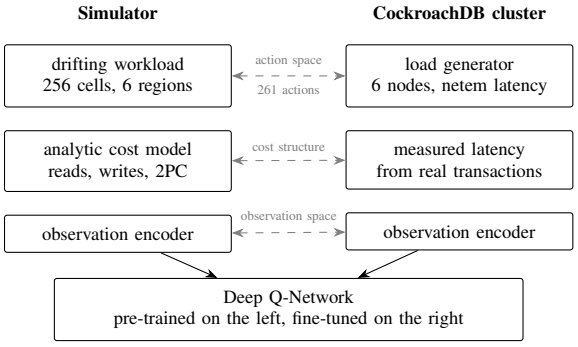


Fig. 1. The two halves of the system and the three things that must agree between them. The agent is pre-trained against the simulator, which is fast and cheap, and fine-tuned against the cluster, which is slow and real. Transfer is only meaningful if the observation space, the action space and the cost structure are the same on both sides, which is why the database was chosen for its ability to satisfy them rather than for its popularity.

I. Validation on a real cluster

The structure of pre-training in a cheap simulator and fine-tuning on expensive real hardware is borrowed from robotics, where it is standard and where the gap between the two is the central difficulty [45]. The difference here is that the gap is narrower and more checkable: the simulator and the cluster can be made to agree on the observation space, the action space and the cost structure by construction, which is why the database was chosen for that property.

The simulator's cost model is an assumption, and the assumption is testable. I deploy six CockroachDB [2] nodes, one per simulated region, in containers on a single host, with wide-area round-trip times injected between them using the Linux kernel's traffic control facility. CockroachDB was selected over sixteen other candidate systems on one criterion: it exposes a *synchronous* statement for relocating a shard's lease and its replicas. Synchrony is essential rather than convenient. If a placement command returns before the placement has changed, the reward observed after an action belongs to a state the action did not produce, and the decision problem is no longer well defined.

IV. Evaluation

The evaluation answers four questions, in order. First, does it actually matter on real hardware where a shard's leaseholder sits, or is the whole premise an artefact of the simulator? Second, is the gain from moving data a gain from *choosing* where to move it, rather than from movement by itself? Third, how does the room available to a better policy change as the

workload moves faster? Fourth, does the learned policy use that room.

The first three have clean answers. The fourth has a partial one, and this section reports the partial answer rather than the one that would read better.

A. Setup

The simulated cluster has six regions, one node per region, replication factor three, and a keyspace divided into 256 cells that shards are built from. An episode is 200 decision windows. At each window the policy sees the per-region read and write counts from the window just finished and issues one action; the environment then charges it for the latency its placement produced and for the data any move had to carry.

Every policy is driven by the same episode loop. The learned policy satisfies the same interface as the baselines and is executed by the same rollout function, so a difference between two numbers in this section cannot be a difference between two pieces of harness code. The baselines are static hash partitioning, random relocation, a reactive heuristic that follows the traffic, and a per-window bound.

The bound deserves a sentence of its own, because it is easy to misread. It is the lowest latency reachable by relocating every lease for free, each window, *at the shard boundaries the episode started with*. It is not a global optimum. A policy that splits a shard can legitimately score below it, so the figures label it "best per window (fixed shards)" rather than "optimal".

The agent minimises a single cost, which is the thing it is being judged on throughout this section:

$$c = \frac{\ell}{100} \;+\; w \cdot \frac{g}{2000} \;+\; 0.10 \cdot b \tag{1}$$

where ℓ is mean transaction latency in milliseconds, g is gigabytes moved over the episode, b is load imbalance, and w is the migration weight. In plain terms: latency is the thing we want low, moving data is charged for, and concentrating the whole workload onto one node is charged for separately so that a policy cannot win on latency by overloading a single region. Migration cost is expressed as a fraction of the dataset rather than in bytes, which is deliberate: a 60 GB simulator and a 1 GB cluster then score the same physical move identically, and a policy trained in one can be evaluated in the other without rescaling the reward.

Two seed blocks appear in this section and they are never mixed in one table. The volatility sweep uses seeds 0 to 7. The agent evaluations use held-out seeds 2000 to 2007, which the training configuration refuses to overlap with its training seeds. A sweep number and an agent number are therefore not directly comparable, and every table states which block it came from.

Each run is identified by the first eight hexadecimal digits of a SHA-256 digest over its resolved configuration. The digest is reconstructible from the parameters alone, which is how the weights behind the runs in Table III were recovered and confirmed without reopening the archives.

The cluster experiments use six CockroachDB nodes in containers with inter-region round-trip latency injected by `netem`, accurate to 0.1 ms across the measured pairs. Both pilots ran on GitHub Actions runners, the first on two cores and the second on four.

TABLE I
CLUSTER PILOTS. TRANSACTION LATENCY WITH LEASEHOLDERS CO-LOCATED AGAINST SPREAD. SOURCE: `RESULTS/RECORDED/CLUSTER.CSV`.

Pilot	Case	p50 (ms)	p95 (ms)	stdev (ms)
$N = 200$, 2 cores	co-located	272.7	275.5	48.8
	spread	1013.4	1648.0	302.0
$N = 1000$, 4 cores	co-located	441.5	446.6	78.4
	spread	1015.3	1648.7	299.9

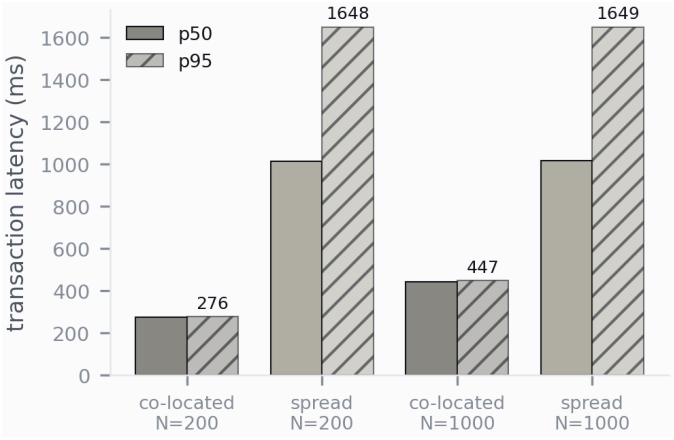


Fig. 2. Co-located against spread leaseholders on the real cluster. The difference is large enough that it is not a measurement artefact, which is all this experiment is asked to establish.

B. Q1: placement changes latency on real hardware

Table I and Fig. 2 report two pilots on the real cluster. In each one the same transactions run twice: once with the leaseholders for the keys involved gathered in the region issuing them, and once with those leaseholders spread across regions.

The p95 gap is 1372.5 ms in the first pilot and 1202.0 ms in the second. Placement is worth a great deal on real hardware, which is the assumption the rest of the work rests on.

Three qualifications belong with that number rather than in a footnote.

First, "co-located" means the leases, not the data. With replication factor three and one node per region, three replicas cannot fit inside one region, so every write still pays a cross-region quorum even in the co-located case. That is why the co-located p50 is 272.7 ms rather than single digits. What the experiment isolates is the leaseholder effect, which is precisely what the MOVE_LEASE action corresponds to.

Second, the interval around p95 is indicative rather than a precision claim. At $N = 200$ transactions the p95 is the 190th order statistic, so a bootstrap interval over it is degenerate at the top. The second pilot raised N to 1000 and halved the interval width from 155.2 ms to 65.6 ms, which is the right

TABLE II
VOLATILITY SWEEP. MEAN LATENCY IN MILLISECONDS AND GIGABYTES MOVED PER EPISODE, EIGHT SEEDS PER VALUE, SEED BLOCK 0 TO 7. SOURCE: RESULTS/RECORDED/SWEEP.CSV.

Volatility	Static	Random	Follow	Bound	Follow GB
0.5	156.27	156.79	94.31	93.87	405.1
1.0	156.11	158.02	98.66	93.66	785.2
2.0	155.37	158.89	125.37	89.47	1056.6
4.0	155.77	160.24	139.58	82.38	1115.2

direction, but choosing N by measurement rather than by convenience remains outstanding work.

Third, and least comfortable, the co-located arm became 62% slower when the pilot moved from two cores to four, 272.7 ms to 441.5 ms, whilst the spread arm barely moved. The co-located case concentrates every lease on one node, and on the larger run that node saturated. The simulator cannot reproduce this, because its cost model has no notion of a node running out of capacity. It is also the clearest evidence in the project that the imbalance term in (1) is measuring something real rather than something invented to make the reward well behaved.

C. Q2: the gain comes from choosing, not from moving

The random baseline exists to separate two explanations that are easy to confuse. It relocates shards at the same rate as the heuristic but to destinations chosen without looking at the traffic. Across all four volatility settings in Table II it moves 204.8 GB and ends up between 156.79 and 160.24 ms, which is no better than doing nothing at all. The heuristic, at volatility 1.0, moves 785.2 GB and reaches 98.66 ms, a 36.8% improvement on static hashing.

So movement by itself buys nothing. Everything in this problem is in the choice of destination. That is worth stating plainly because it is the reason the problem is a decision problem at all, and therefore the reason a learned policy is a sensible thing to attempt.

D. Q3: the room for a better policy grows with volatility

Volatility controls how fast the geographic centre of the workload drifts between windows. Table II and Fig. 3 sweep it over a factor of eight. Three things happen at once, and the third is the finding.

Static hashing is flat, 155.37 to 156.27 ms across the whole sweep. It does not degrade as the workload moves faster because it was never tracking the workload in the first place. Anything it does badly, it does badly at every setting.

The bound *improves* as volatility rises, 93.87 ms down to 82.38 ms. This is initially counter-intuitive and worth a sentence. A more volatile workload is a more sharply concentrated one, and a sharply concentrated workload is easier to serve well if you put the data in the right place. Faster drift therefore makes the problem harder for a policy and easier for an oracle.

The heuristic goes the other way. Its latency rises from 94.31 to 139.58 ms whilst the data it moves rises from 405.1 to 1115.2 GB (Fig. 4). It pays nearly three times as much and

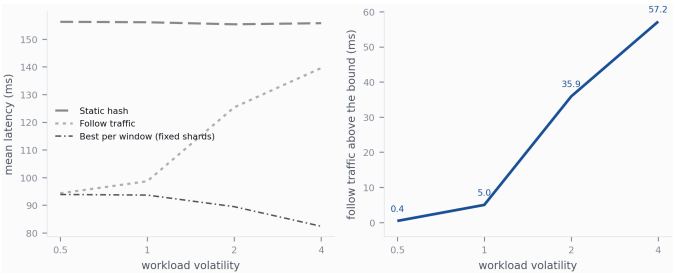


Fig. 3. Left: latency against volatility for static hashing, the reactive heuristic, and the per-window bound. Right: how far the heuristic sits above the bound. The gap is the room available to a better policy, and it opens from 0.44 ms to 57.20 ms. Seed block 0 to 7.

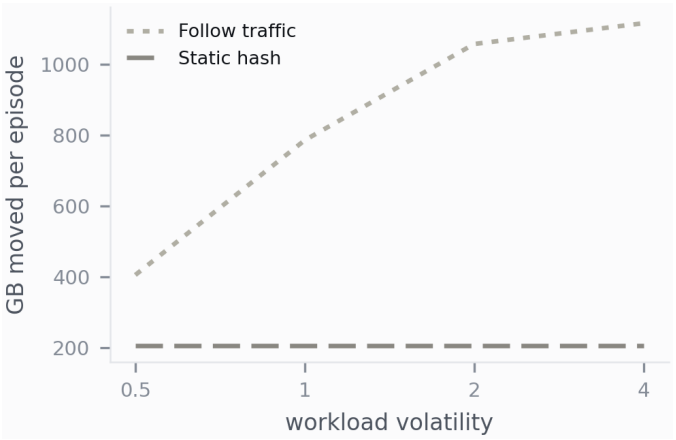


Fig. 4. Data moved per episode. The heuristic's bill rises with volatility whilst its advantage falls, which is the behaviour a learned policy has to improve on.

gets progressively less for it. A rule that reacts to the window just finished is chasing a target that has already moved on, and under fast drift most of what it buys with that movement has expired by the time the move lands.

The consequence is the right-hand panel of Fig. 3. The gap between the heuristic and the bound is 0.44 ms at volatility 0.5 and 57.20 ms at volatility 4, which is 41% of the heuristic's own latency. Put simply: under slow drift a reactive rule is already close to the best anyone could do, and there is nothing worth learning. Under fast drift there is a great deal worth learning. The case for a learned policy is conditional on volatility, and that condition is now measured rather than assumed.

This also corrects an earlier reading of our own data. From the volatility 1.0 results alone, where the gap is 5.00 ms, we concluded that latency was not a viable contribution and that the work would have to argue from movement cost instead. The sweep shows that conclusion was drawn from a single point on a curve that had not been plotted.

E. Q4: the learned policy wins at high volatility, narrowly

Table III reports five agent runs on held-out seeds 2000 to 2007, each scored against the baselines under (1) at that run's

TABLE III
LEARNED POLICY AGAINST THE BASELINES, HELD-OUT SEEDS 2000 TO 2007. COSTS ARE FROM (1), EACH PAIR SCORED AT THE SAME w. NEGATIVE MEANS THE AGENT IS BETTER. SOURCE: RESULTS/RECORDED/AGENT-RUNS.CSV, SCORED BY SCRIPTS/SCORE_OBJECTIVE.PY.

Run	Vol	w	Seed	ms	GB	vs follow	vs static
3	4.0	0.30	0	139.91	62.7	−0.068	−0.071
7	4.0	0.30	1	144.61	44.5	−0.031	−0.035
4	4.0	0.10	0	136.94	123.7	+0.012	−0.100
8	2.0	0.30	0	129.64	177.3	+0.003	−0.167
6	2.0	0.00	0	129.24	311.9	+0.124	−0.202

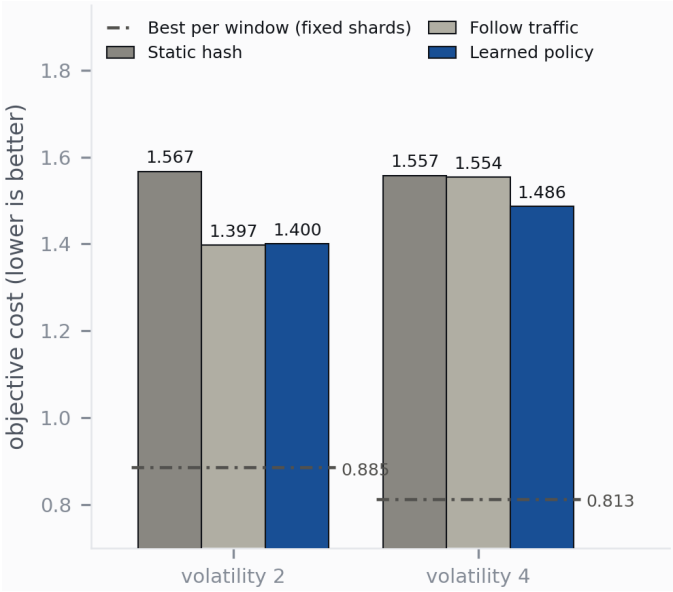


Fig. 5. Every policy scored under the full objective at $w = 0.30$, not on latency alone. At volatility 4 the heuristic's latency advantage is almost entirely paid back by the data it moves and the imbalance it creates, and the learned policy is below both. The distance still remaining to the bound is the honest measure of how much is left to do.

own migration weight. Fig. 5 shows the same comparison for the $w = 0.30$ runs.

The agent beats static hashing in all five runs. At volatility 4 with $w = 0.30$ it also beats the heuristic, on both learner seeds, by 0.068 and 0.031 of objective cost. It does so in a way that is easy to describe: it reaches 139.91 ms against the heuristic's 129.04 ms, so it is slower, but it moves 62.7 GB against 1086.5 GB and holds imbalance at 0.776 against 1.0034. It is buying its latency far more cheaply than the heuristic buys its own.

Fig. 5 shows a second result that falls out of the same scoring. At volatility 4 the heuristic and static hashing are indistinguishable under the objective, 1.554 against 1.557, despite the heuristic being 21 ms faster. Everything it gains on latency it returns in movement and imbalance. The reactive rule has stopped being worth running at all by that point, which is a sharper statement of Q3's finding than the latency curve alone can make.

Four qualifications, and they are substantial.

The policy has not converged. Held-out evaluation was still improving at the final training step in every run. The training budget ran out before the learning did, so these are lower bounds on what this architecture reaches, not its ceiling.

No claim about the migration weight is supportable from this data. The two learner seeds at volatility 4 differ by 4.70 ms, whilst changing w from 0.30 to 0.10 at a fixed seed changes latency by 2.97 ms. The spread between seeds is larger than the effect of the treatment, so runs 3, 4 and 7 cannot be read as telling us what w should be. More seeds per setting is the fix, and it has not been done.

Most of the agent's actions do nothing. Between 80% and 90% of the actions it selects are inapplicable in the state it selected them in, which the environment resolves as a no-op. The random baseline sits at 80.4% on the same measure, so the learned policy is barely better than chance at choosing an action that is even legal. It is winning on the actions that do land, and it is wasting most of its decisions. This is the single largest measured inefficiency in the project.

The bound is still far away. At volatility 4 the agent scores 1.486 against the bound's 0.813. Beating a heuristic that has itself stopped working is a much weaker result than closing the gap to what is achievable, and Fig. 5 is drawn so that the remaining distance is visible rather than cropped out.

F. Threats to validity

The simulator's cost model is analytic. Q1 establishes that placement matters on real hardware; it does not establish that the simulator's magnitudes are right, and no claim in this paper should be read as if it did.

The two pilots differ in both transaction count and core count, so the difference between them cannot be attributed to either one alone. They are reported as two separate pilots for that reason.

N was chosen by convenience in the first pilot and raised in the second without a power calculation behind either value.

The bound holds the starting shard boundaries fixed, so it is not a global optimum and a policy that splits shards may legitimately pass it.

The high rate of inapplicable actions is a known failure mode as the invalid share of an action space grows [46], and action-elimination networks built on DQN address it directly [47]. Neither is implemented here.

Results at one volatility should not be generalised to another, which is exactly what Q3 demonstrates. The agent results at volatility 4 say nothing about volatility 1, and the project's own earlier misreading is the cautionary case.

V. CONCLUSION AND FUTURE WORK

This paper set out to test whether a reinforcement learning agent can act as an autonomous database administrator, re-fragmenting and re-allocating shards in real time as a workload's geographic centre of gravity moves. What has been established is a measured problem statement, a cluster that

confirms the premise, and a first learned policy that wins where the problem is hardest.

The problem is stated as a Markov decision process with an action encoding that was measured rather than assumed, four baselines running through one shared episode loop, and a reward in which migration cost is a fraction of the dataset rather than a byte count, so that a policy trained in the simulator can be evaluated on the cluster without the reward changing meaning underneath it. On the real cluster, gathering leaseholders in the region issuing the transactions against spreading them across regions changes p95 latency by 1372.5 ms. Placement matters, measurably, on hardware.

The main result is about when a learned policy is worth having. A reactive heuristic that follows the traffic is 0.44 ms off the achievable bound under slow drift and 57.20 ms off it under fast drift, which is 41% of its own latency, whilst the data it moves rises from 405.1 GB to 1115.2 GB per episode. It pays nearly three times as much and gets progressively less. Scored under the full objective at volatility 4 it is indistinguishable from static hashing, 1.554 against 1.557, because everything it gains on latency it hands back in movement and imbalance. The case for learning is therefore conditional on volatility, and this work measures the condition instead of asserting it.

Against that heuristic, the learned policy wins at volatility 4 on both learner seeds, by 0.068 and 0.031 of objective cost, and beats static hashing in every run recorded. It does so by being slower and very much cheaper: 139.91 ms against 129.04 ms, but 62.7 GB moved against 1086.5 GB. That is a narrow win over a heuristic that has itself stopped working, and it should be read as such. The policy sits at 1.486 against a bound of 0.813, it had not converged when the training budget ran out, and between 80% and 90% of the actions it selects are inapplicable in the state it selects them in. The agent is winning on the minority of its decisions that land.

A. Future work

In the order they should be attempted.

Fix the action space first. An 80% to 90% inapplicable rate is the largest measured inefficiency in the project, it is a known failure mode as the invalid share of an action space grows [46], and action-elimination networks built on DQN address exactly it [47]. Nothing else on this list is likely to matter as much.

Run more seeds per setting. The spread between two learner seeds currently exceeds the effect of changing the migration weight, which means the present data cannot answer how that weight should be set. This is cheap and it blocks a question the work already wants to ask.

Address credit assignment. A move is charged in full at the moment it is made whilst the latency it buys arrives over later windows, which is a disagreement a Q-function has to resolve from a short training budget. n-step returns, as combined in Rainbow [40], or a shaping term that credits a move with the latency it later saves, are the two obvious routes.

Train for longer. Every run was still improving at its final step, so the reported numbers are a floor rather than a ceiling. Choose the cluster transaction count by measurement rather than by convenience, and separate the transaction count from the core count so that the saturation effect seen between the two pilots can be attributed.

Fine-tune the pre-trained agent on the real cluster. The apparatus exists, the reward normalisation that makes transfer meaningful is already in place, and the harness that would drive it is not yet built.

Add a cost-model rebalancer in the style of Lion [23] as a stronger baseline. Following the traffic is a weak opponent at volatility 4, and the next version of this result should be measured against something that is still working.

References

[1] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013.

[2] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, "CockroachDB: The resilient geo-distributed SQL database," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1493–1509, 2020.

[3] S. Ceri, M. Negri, and G. Pelagatti, "Horizontal data partitioning in database design," in *Proceedings of the 1982 ACM SIGMOD International Conference on Management of Data*, pp. 128–136, 1982.

[4] S. Navathe, S. Ceri, G. Wiederhold, and J. Dou, "Vertical partitioning algorithms for database design," *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 680–710, 1984.

[5] P. M. G. Apers, "Data allocation in distributed database systems," *ACM Transactions on Database Systems*, vol. 13, no. 3, pp. 263–304, 1988.

[6] D. Saccà and G. Wiederhold, "Database partitioning in a cluster of processors," *ACM Transactions on Database Systems*, vol. 10, no. 1, pp. 29–56, 1985.

[7] S. T. March and S. Rho, "Allocating data and operations to nodes in distributed database design," *IEEE Transactions on Knowledge and Data Engineering*, vol. 7, no. 2, pp. 305–317, 1995.

[8] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Springer, 3 ed., 2011. Crossref's bibliographic query did not surface the monograph record; the DOI is the Springer book DOI and was not confirmed against a Crossref work record.

[9] D. Nashat and A. A. Amer, "A comprehensive taxonomy of fragmentation and allocation techniques in distributed database design," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–25, 2018.

[10] A. A. Amer, A. A. Sewisy, and T. M. A. Elgendy, "An optimized approach for simultaneous horizontal data fragmentation and allocation in distributed database systems (DDBSs)," *Heliyon*, vol. 3, no. 12, p. e00487, 2017.

[11] P. Niu, B. Ren, H. Wu, and X. Yao, "Modeling and optimization for massive data allocation in database." arXiv preprint, 2026. Preprint, not peer reviewed at the time of citing. Cited as a current instance of the classical allocation formulation, not as an established result.

[12] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the world wide web," in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing (STOC)*, pp. 654–663, 1997. Crossref's title field holds only "Consistent hashing and random trees"; the subtitle here is from the published paper.

[13] S. Weil, S. Brandt, E. Miller, and C. Maltzahn, "CRUSH: Controlled, scalable, decentralized placement of replicated data," in *ACM/IEEE SC 2006 Conference (SC'06)*, p. 31, 2006. A duplicate ACM record exists under 10.1145/1188455.1188582 with a mangled title; the IEEE record is used.

[14] M. Coluzzi, A. Brocco, P. Contu, and T. Leidi, "A survey and comparison of consistent hashing algorithms," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 346–348, 2023.

[15] C. Curino, E. Jones, Y. Zhang, and S. Madden, "Schism: a workload-driven approach to database replication and partitioning," *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 48–57, 2010.

[16] A. Pavlo, C. Curino, and S. Zdonik, "Skew-aware automatic database partitioning in shared-nothing, parallel OLTP systems," in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pp. 61–72, 2012.

[17] R. Taft, E. Mansour, M. Serafini, J. Duggan, A. J. Elmore, A. Aboulnaga, A. Pavlo, and M. Stonebraker, "E-Store: Fine-grained elastic partitioning for distributed transaction processing," *Proceedings of the VLDB Endowment*, vol. 8, no. 3, pp. 245–256, 2014.

[18] M. Serafini, R. Taft, A. J. Elmore, A. Pavlo, A. Aboulnaga, and M. Stonebraker, "Clay: Fine-grained adaptive partitioning for general database schemas," *Proceedings of the VLDB Endowment*, vol. 10, no. 4, pp. 445–456, 2016.

[19] J. Kamal, M. Murshed, and R. Buyya, "Workload-aware incremental repartitioning of shared-nothing distributed databases for scalable OLTP applications," *Future Generation Computer Systems*, vol. 56, pp. 421–435, 2016.

[20] Z. Ding, X. Zhang, W. Lu, W. Ma, W. Zhang, and X. Du, "Promi: Progressive live migration in distributed database systems," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pp. 349–362, 2025.

[21] A. Atrey, G. V. Seghbroeck, H. Mora, F. D. Turck, and B. Volckaert, "Unifying data and replica placement for data-intensive services in geographically distributed clouds," in *Proceedings of the 9th International Conference on Cloud Computing and Services Science (CLOSER)*, pp. 25–36, 2019.

[22] A. H. Adnan, A. M. A. Al-Muqarm, and A. S. Abosinnee, "Network-aware optimization for efficient data placement in geo-distributed cloud systems," *Journal of Grid Computing*, vol. 23, 2025.

[23] Q. Zheng, Z. Zhao, W. Lu, C. Yao, Y. Chen, A. Pan, and X. Du, "Lion: Minimizing distributed transactions through adaptive replica provision," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pp. 2012–2025, 2024. Read from the PDF. Extended version: arXiv:2403.11221.

[24] Q. Zhuang, X. Shi, S. Liu, W. Lu, Z. Zhao, Y. Chen, T. Li, A. Pan, and X. Du, "GeoTP: Latency-aware geo-distributed transaction processing in database middlewares," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pp. 433–445, 2025. Read from the PDF.

[25] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu, M. Ran, and Z. Li, "An end-to-end automatic cloud database tuning system using deep reinforcement learning," in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pp. 415–432, 2019.

[26] G. Li, X. Zhou, S. Li, and B. Gao, "QTune: A query-aware database tuning system with deep reinforcement learning," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2118–2130, 2019.

[27] X. Zhao, X. Zhou, and G. Li, "Automatic database knob tuning: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12470–12490, 2023.

[28] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019.

[29] J. Wang, I. Trummer, and D. Basu, "UDO: Universal database optimization using reinforcement learning," *Proceedings of the VLDB Endowment*, vol. 14, no. 13, pp. 3402–3414, 2021.

[30] K. Lu, N. Zhao, J. Wan, C. Fei, W. Zhao, and T. Deng, "RLRP: High-efficient data placement with reinforcement learning for modern distributed storage systems," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 595–605, 2022.

[31] G. Singh, R. Nadig, J. Park, R. Bera, N. Hajinazar, D. Novo, J. Gómez-Luna, S. Stuijk, H. Corporaal, and O. Mutlu, "Sibyl: adaptive and extensible data placement in hybrid storage systems using online reinforcement learning," in *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pp. 320–336, 2022.

[32] R. Nadig, V. Arulchelvan, R. Bera, T. Shahroodi, G. Singh, A. K. Kakolyris, İ. E. Yüksel, M. Sadrosadati, J. Park, and O. Mutlu, "Harmonia: Enhancing data placement and migration in hybrid storage systems via multi-agent reinforcement learning," in *Proceedings of the 40th ACM International Conference on Supercomputing (ICS)*, pp. 251–271, 2026. Crossref records 2026; the Consensus index lists 2025. The Crossref year is used here.

[33] M. Zheng, X. Du, Z. Lu, and Q. Duan, "A balanced and reliable data replica placement scheme based on reinforcement learning in edge–cloud environments," *Future Generation Computer Systems*, vol. 155, pp. 132–145, 2024.

[34] Z. Liao, R. Zhang, S. He, D. Zeng, J. Wang, and H.-J. Kim, "Deep learning-based data storage for low latency in data center networks," *IEEE Access*, vol. 7, pp. 26411–26417, 2019.

[35] S. H. Mahmoud, N. Badr, A. E. Abdelraouf, and M. I. Ali, "RL-based fragment allocation and replication for distributed heritage multimedia databases," *International Journal of Intelligent Computing and Information Sciences*, vol. 25, pp. 73–92, 2025.

[36] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pp. 143–154, 2010.

[37] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. No DOI. JMLR does not register Crossref DOIs, so this entry could not be verified against Crossref. Metadata taken from the JMLR landing page.

[38] M. Towers, A. Kwiatkowski, J. Balis, G. D. Cola, T. Deleu, M. Goulão, K. Andreas, M. Krimmel, A. KG, R. Perez-Vicente, J. Terry, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," 2025.

[39] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[40] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.

[41] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 2016.

[42] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning." arXiv:1511.06581, 2015. Published at ICML 2016. No Crossref record: PMLR proceedings carry no DOIs. Metadata from the arXiv abstract page.

[43] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay." arXiv:1511.05952, 2015. Published at ICLR 2016. No Crossref record: ICLR carries no DOIs. Metadata from the arXiv abstract page.

[44] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[45] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.

[46] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," *The International FLAIRS Conference Proceedings*, vol. 35, 2022.

[47] T. Zahavy, M. Haroush, N. Merlis, D. J. Mankowitz, and S. Mannor, "Learn what not to learn: Action elimination with deep reinforcement learning." arXiv:1809.02121, 2018. Published at NeurIPS 2018. No Crossref record: NeurIPS proceedings carry no DOIs. Metadata from the arXiv page.